

INFORME DE PROYECTO

To Do List

Aplicación móvil Android — React Native + Expo

Examen 2 — Aplicaciones Móviles

Campo	Detalle
Autor	Ignacio Billeke
Plataforma objetivo	Android
Stack	React Native · Expo SDK 54 · TypeScript
Android package	com.ignac.todolist
Gestor de paquetes	pnpm
Fecha del informe	16 de agosto de 2026

Índice

1. Resumen del proyecto
2. Objetivos y alcance
3. Stack tecnológico
4. Modelo de datos
5. Arquitectura y estructura del proyecto
6. Pantallas y navegación
7. Diseño visual (UI/UX)
8. Funcionalidades implementadas
9. Evidencia de pruebas: permisos de cámara y ubicación
10. Capturas adicionales de la aplicación
11. Pruebas automatizadas (Jest)
12. Instalación y ejecución
13. Roadmap (fuera de alcance actual)
14. Conclusión

1. Resumen del proyecto

To Do List es una aplicación móvil desarrollada en React Native + Expo (SDK 54) con TypeScript, orientada a la plataforma push y Android. Permite a cada usuario registrar y visualizar sus tareas pendientes, adjuntando de forma opcional una foto (tomada con la cámara del dispositivo) y la ubicación GPS del lugar donde se creó la tarea.

La aplicación no depende de un backend propio: la autenticación es local (usuarios y contraseñas hasheadas guardados en el dispositivo) y la persistencia de datos se realiza con AsyncStorage + expo-file-system. Adicionalmente, integra la API pública JSONPlaceholder para importar tareas de ejemplo y demostrar sincronización saliente vía POST.

Objetivos cumplidos

- **Interacción con periféricos:** cámara y GPS, con solicitud de permisos lazy.
- **Autenticación de usuarios:** registro y login locales, sin backend propio.
- **Integración con servicio web externo:** JSONPlaceholder (importación + sincronización).
- **Pruebas automatizadas:** Jest, mockeando cámara y ubicación.

Principios de diseño

- Permisos solicitados de forma lazy — nunca al abrir la app, sólo al usar la funcionalidad.
- Ningún permiso rechazado bloquea el flujo de guardado de una tarea.
- Almacenamiento local (AsyncStorage) como única fuente de verdad de los datos.
- Identidad visual moderna y amigable: violeta + coral sobre fondo crema cálido.

2. Objetivos y alcance

El alcance del proyecto se organiza en cuatro ejes principales, definidos en docs/BRIEF.md:

2.1 Interacción con periféricos (cámara y GPS)

- Capturar imágenes con la cámara y adjuntarlas a una tarea — campo opcional (photoUri).
- Registrar la ubicación (GPS) donde se crea la tarea — campo opcional (location).
- Solicitud lazy de permisos, recién al tocar "Agregar foto" / "Usar ubicación actual".

- Si el usuario rechaza el permiso, se cancela sólo esa acción puntual; la tarea se crea igual, sin ese dato.

2.2 Autenticación de usuarios

- Login y registro locales, sin backend propio.
- Alta de cuenta: usuario único (case-insensitive) + contraseña hasheada (SHA-256 vía expo-crypto).
- Sesión persistida en AsyncStorage entre reinicios, hasta cierre de sesión explícito.
- Multi-usuario en el mismo dispositivo; cada tarea pertenece a su usuario (Task.userId).

2.3 Integración con servicios web y APIs

- Importación de tareas desde JSONPlaceholder (GET /todos) para poblar la lista inicial.
- Sincronización saliente de tareas locales hacia JSONPlaceholder vía POST /todos.
- JSONPlaceholder no persiste escrituras server-side — AsyncStorage sigue siendo la fuente de verdad real.

2.4 Pruebas automatizadas

- Framework Jest + preset jest-expo, mockeando expo-camera y expo-location.
- Cobertura mínima: captura de imágenes y obtención de ubicación GPS.
- Cobertura adicional: flujo de autenticación e integración con JSONPlaceholder.

3. Stack tecnológico

Área	Decisión
Frontend	React Native + Expo Go, TypeScript
Expo SDK	54 (fijo — ~54.0.0)
Plataforma objetivo	Android
Android package	com.ignac.todolist
Backend	Ninguno — Node.js sólo como tooling de desarrollo (Expo CLI, pnpm)

Área	Decisión
Persistencia	Local — AsyncStorage + expo-file-system
API externa	JSONPlaceholder (importación + sincronización de tareas vía POST)
Navegación	React Navigation (Stack Navigator)
Testing	Jest + jest-expo
Gestor de paquetes	pnpm (obligatorio, no npm/yarn)

Dependencias principales

Paquete	Uso
expo-camera	Captura de fotos
expo-location	Obtención de coordenadas GPS
expo-crypto	UUID (randomUUID) y hash de contraseña (SHA-256, digestStringAsync)
expo-file-system	Guardado de fotos en el filesystem del dispositivo
expo-image-manipulator	Resize/compresión de fotos (máx. 1080px, calidad 70%)
@react-native-async-storage/async-storage	Persistencia local (usuarios, sesión, tareas)
@react-navigation/native + native-stack	Navegación entre pantallas
@expo/vector-icons	Iconografía (ubicación, estados, etc.)
jest + jest-expo	Pruebas automatizadas

4. Modelo de datos

User

```

type User = {
  id: string;          // uuid generado localmente
  username: string;    // único, case-insensitive
  passwordHash: string; // SHA-256 (expo-crypto) — nunca texto plano
  createdAt: string;   // ISO date
};

```

Task

```

type Task = {
  id: string;          // uuid generado localmente
  userId: string;      // dueño de la tarea (User.id), obligatorio
  title: string;       // obligatorio
  description?: string; // opcional
  completed: boolean;
  createdAt: string;   // ISO date
  photoUri?: string;   // path local del archivo (filesystem)
  location?: { latitude: number; longitude: number };
  source: "local" | "jsonplaceholder";
  syncedAt?: string;   // ISO date de la última sincronización exitosa
};

```

AsyncStorage guarda además la lista completa de `User[]` y una clave de sesión (`currentUserId: string | null`).

5. Arquitectura y estructura del proyecto

El código se organiza en dos carpetas de primer nivel: `docs/` (documentación del proyecto) y `app/` (código fuente Expo/React Native), separadas de forma explícita.

```

Examen2AppMovil/
├── docs/          # Brief, diseño, plan y pruebas
│   ├── BRIEF.md
│   ├── DESIGN.md
│   ├── PLAN.md
│   ├── INFORME_PROYECTO.pdf
│   └── pruebas/   # Evidencia de permisos (capturas)

```

```

└─ app/                # Código fuente Expo/React Native
  └─ App.tsx           # Entry point, providers, stack raíz
  └─ index.ts
  └─ app.json          # Config Expo (permisos Android, plugins)
  └─ src/
    └─ api/            # Cliente JSONPlaceholder
    └─ auth/           # AuthContext (sesión)
    └─ components/     # AuthHero, ScreenHeader, TaskCard
    └─ navigation/     # Tipos de navegación
    └─ screens/        # Login, Register, TaskList, TaskForm
    └─ storage/        # authStorage, taskStorage (AsyncStorage)
    └─ theme/          # Colores y estilos
    └─ types/          # Task, User
  └─ __tests__/        # Tests Jest

```

Configuración de permisos nativos (app.json)

Los permisos de cámara y ubicación se declaran mediante los plugins de configuración de Expo, con textos de justificación en español mostrados por el sistema operativo:

```

"plugins": [
  ["expo-camera", {
    "cameraPermission": "Permitir que $(PRODUCT_NAME) acceda a tu
                        cámara para adjuntar fotos a las tareas."
  }],
  ["expo-location", {
    "locationWhenInUsePermission": "Permitir que $(PRODUCT_NAME) acceda
                                   a tu ubicación para adjuntarla a las tareas."
  }]
]

```

6. Pantallas y navegación

Navegación implementada con React Navigation (Stack Navigator). Dos stacks se montan de forma condicional según haya o no una sesión activa (currentUserId en AsyncStorage):

AuthStack (sin sesión)

- |— LoginScreen — usuario, contraseña, botón "Ingresar", link a registro
- |— RegisterScreen — usuario, contraseña, confirmar contraseña

MainStack (con sesión)

- |— TaskListScreen — lista de tareas (FlatList), FAB "+",
| Importar / Sincronizar, cerrar sesión
- |— TaskFormScreen — crear / editar / ver tarea: título, descripción,
foto, ubicación, completada

7. Diseño visual (UI/UX)

Dirección visual: moderna y amigable — violeta + coral sobre fondo cálido, bordes redondeados y espaciado generoso.

Paleta de colores

Uso	Color
Primario (acciones, header, FAB)	#7C3AED — violeta
Acento secundario (badges)	#FF6B6B — coral
Fondo	#FFFBF5 — crema cálido
Superficie / tarjeta	#FFFFFF
Texto principal	#292524
Texto secundario	#78716C
Éxito / check	#22C55E
Error / eliminar	#EF4444

Componentes clave

- **TaskCard:** checkbox, título, thumbnail de foto, ícono de ubicación, ícono de sincronizado, texto tachado si completada.
- **FAB:** botón flotante "+" (violeta), esquina inferior derecha, abre TaskFormScreen en modo crear.

- **AuthHero:** franja violeta superior con chips animados estilo TaskCard, wordmark y tagline, sobre una hoja crema con esquinas redondeadas.
- **TaskForm:** título, descripción, "Agregar foto", "Usar ubicación actual", switch "Completada", botones Guardar/Eliminar.

Forma: borderRadius 12–16, sombra suave en tarjetas (elevation 2 / shadowOpacity 0.08), padding 16. Tipografía: system default (Roboto en Android).

8. Funcionalidades implementadas

8.1 Autenticación local

- Registro con usuario único (case-insensitive) y contraseña hasheada con SHA-256 (expo-crypto).
- Login compara el hash guardado; en error nunca revela cuál dato falló ("Usuario o contraseña incorrectos").
- Sesión persistida en AsyncStorage; se mantiene entre reinicios hasta cerrar sesión explícitamente.
- Multi-usuario en el mismo dispositivo; cada tarea queda asociada a su dueño (Task.userId).

8.2 Cámara y ubicación (periféricos)

- Permisos solicitados de forma lazy: recién al tocar "Agregar foto" / "Usar ubicación actual".
- Foto: capturada con expo-camera, comprimida con expo-image-manipulator (máx. 1080px, calidad 70%) y guardada en el filesystem del dispositivo (expo-file-system); sólo se persiste el path en AsyncStorage, nunca el binario.
- Ubicación: obtenida con expo-location; una vez agregada, se muestra "Ubicación agregada ✓" en vez de coordenadas crudas.
- Rechazo de permiso (en el momento o permanente): cancela sólo esa acción puntual; la tarea se guarda igual, sin ese dato.

8.3 Integración con JSONPlaceholder

- Importación inicial: GET jsonplaceholder.typicode.com/todos; tareas marcadas con source: "jsonplaceholder", sin duplicar en reimportaciones.

- Sincronización saliente: botón "Sincronizar" hace POST /todos por cada tarea local pendiente; al recibir 201 se marca syncedAt.
- JSONPlaceholder no persiste escrituras server-side — la fuente de verdad real es siempre AsyncStorage en el dispositivo.

8.4 Estados de UI

- Vacío: ícono + "No tenés tareas todavía" + botones "Crear tu primera tarea" / "Importar tareas".
- Cargando: ActivityIndicator durante importación, sincronización o guardado de foto.
- Error de red: mensaje corto + botón "Reintentar"; las tareas locales ya guardadas siguen visibles y usables.

9. Evidencia de pruebas: permisos de cámara y ubicación

Pruebas realizadas sobre emulador Android (Pixel 7, Android 14, API 34, imagen google_apis_playstore), app "To Do List" corriendo vía Expo Go (SDK 54).

Flujo probado: TaskListScreen ("Mis tareas") → FAB "+" → TaskFormScreen → botones "Agregar foto" / "Usar ubicación actual".

9.1 Permiso de cámara

9.1.1 Solicitud lazy del permiso

Al tocar "Agregar foto" por primera vez, Android muestra el diálogo nativo de permiso recién en ese momento (no al abrir la app), tal como especifica el flujo de permisos de docs/DESIGN.md.

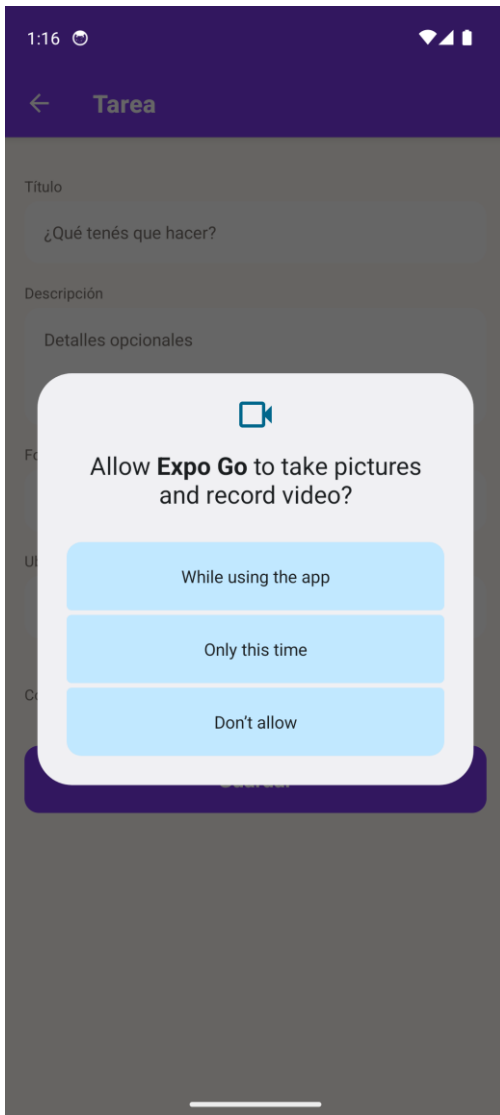


Fig. 1 — Diálogo nativo de permiso de cámara ("Allow Expo Go to take pictures and record video?")

9.1.2 Permiso otorgado

Al elegir "While using the app", el permiso se concede y se abre la vista de cámara (CameraView) con el botón de captura.

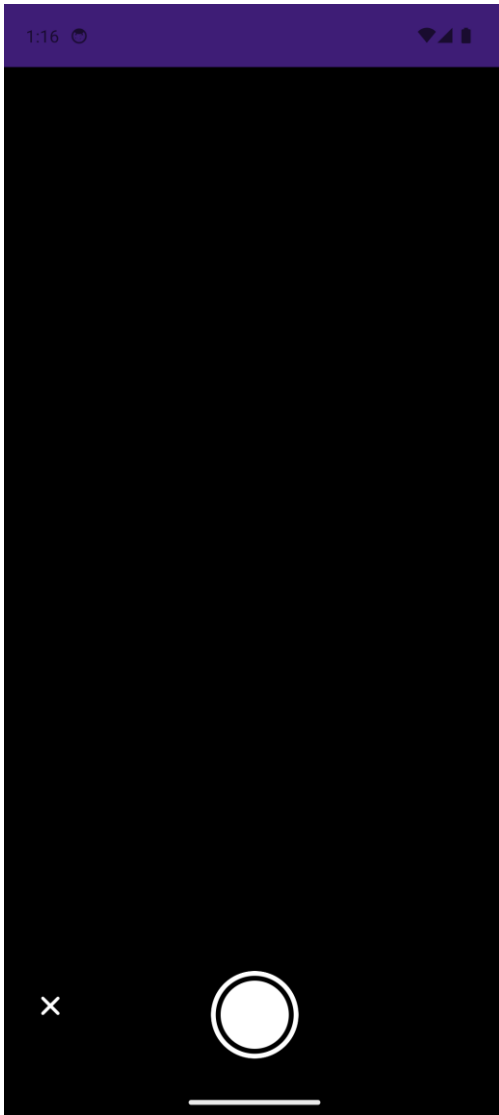
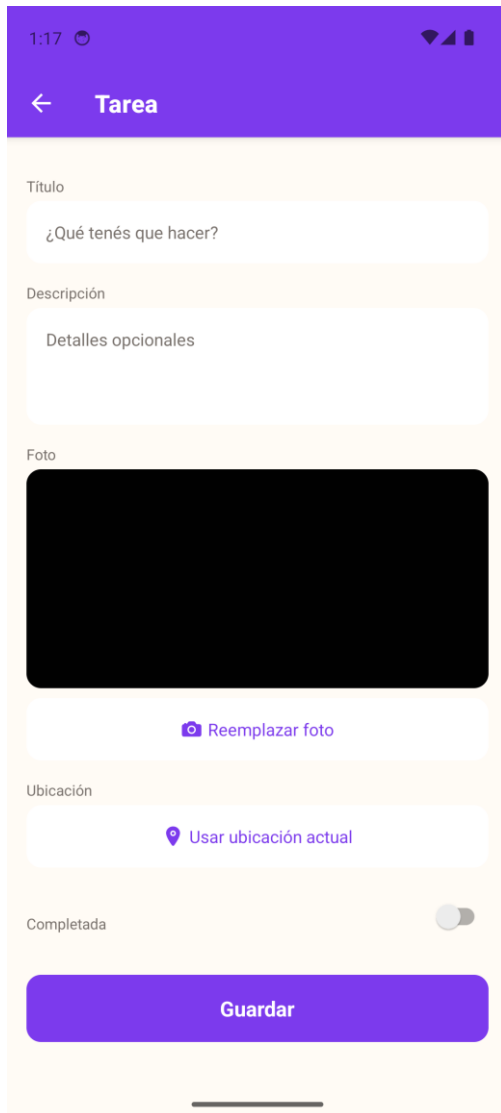


Fig. 2 — Permiso de cámara otorgado; vista de cámara abierta

9.1.3 Foto capturada y guardada

Tras tocar el botón de captura, la foto se comprime (expo-image-manipulator, máx. 1080px, calidad 70%), se guarda en el filesystem del dispositivo (expo-file-system) y el formulario muestra el preview con el botón cambiado a "Reemplazar foto", confirmando que photoUri quedó seteado.



1:17

← Tarea

Título

¿Qué tenés que hacer?

Descripción

Detalles opcionales

Foto

Reemplazar foto

Ubicación

Usar ubicación actual

Completada

Guardar

Fig. 3 — Foto capturada y guardada; preview visible en el formulario

9.2 Permiso de ubicación

9.2.1 Solicitud lazy del permiso

Al tocar "Usar ubicación actual" por primera vez, Android muestra el diálogo nativo pidiendo acceso a la ubicación (con opciones Precise/Approximate), también solicitado de forma lazy.

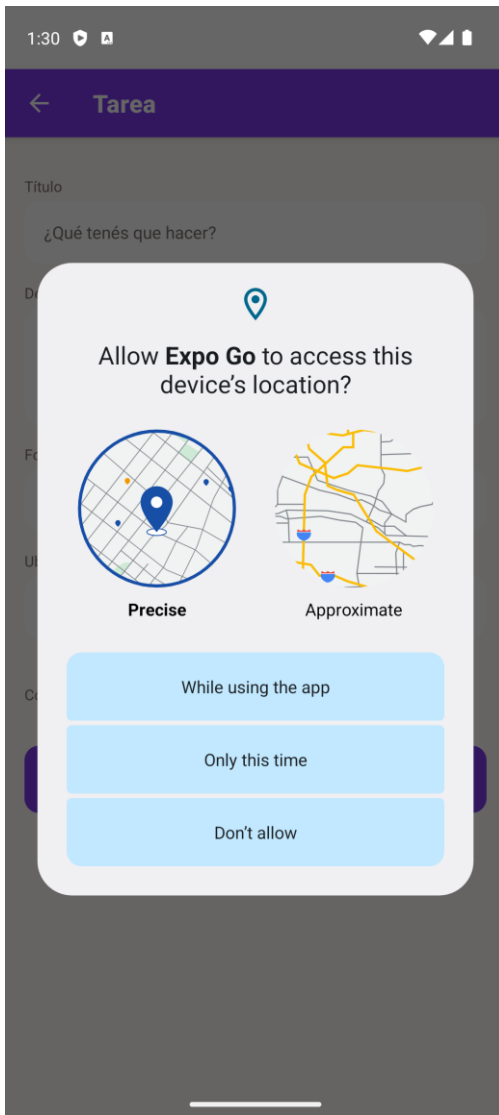


Fig. 4 — Diálogo nativo de permiso de ubicación (Precise / Approximate)

9.2.2 Permiso otorgado

Al elegir "While using the app", el permiso se concede — confirmado además a nivel de sistema operativo (logcat: `PermissionGrantResult ... permission=android.permission.ACCESS_FINE_LOCATION ... result=4, grant real, no simulado`). La app entra en estado de carga (`isFetchingLocation`) mientras espera la coordenada del sistema.

1:30

Tarea

Título

¿Qué tenés que hacer?

Descripción

Detalles opcionales

Foto

Agregar foto

Ubicación

C

Completada

Guardar

Fig. 5 — Permiso de ubicación otorgado; formulario en estado de carga

Nota: la resolución final de la coordenada GPS quedó bloqueada por una limitación conocida del Fused Location Provider del emulador Android (no procesa correctamente los fixes inyectados por consola sin un dispositivo físico o Extended Controls por GUI). El permiso en sí — objeto de esta prueba — quedó otorgado y confirmado por el sistema operativo; el código de la app (TaskFormScreen.tsx, handleUseLocation) no arrojó ningún error durante la prueba.

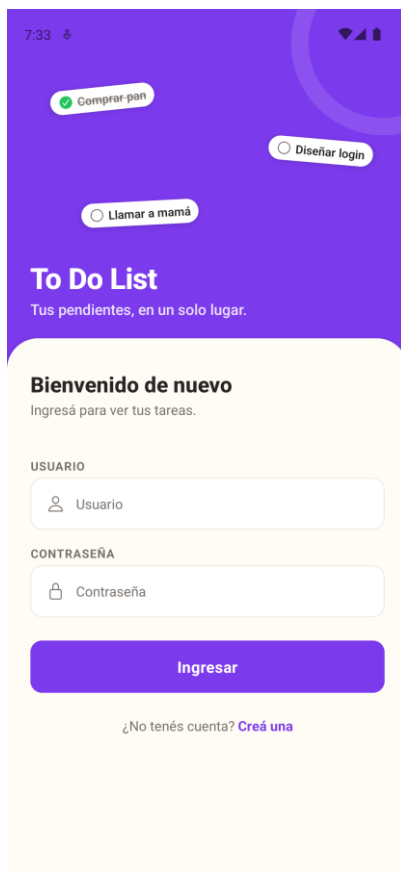
9.3 Resumen de resultados

Permiso	Solicitud lazy	Diálogo nativo	Permiso otorgado	Acción posterior
Cámara	✓	✓	✓	✓ Foto capturada y guardada

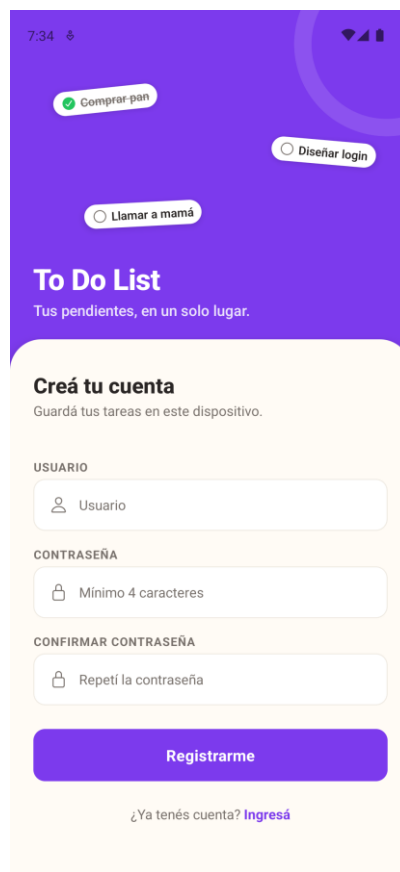
Permiso	Solicitud lazy	Diálogo nativo	Permiso otorgado	Acción posterior
Ubicación	✓	✓	✓ (confirmado por SO)	⚠ Coordinada bloqueada por el emulador (no por la app)

10. Capturas adicionales de la aplicación

Además de la evidencia de permisos, se incluyen capturas generales de la aplicación corriendo en emulador Android, mostrando el flujo de autenticación, el formulario de tareas y el estado sincronizado de la lista.



Login — pantalla de inicio de sesión (diseño final)



Registro — creación de cuenta (diseño final)

7:38

← Tarea

Título

Comprar materiales examen

Descripción

Revisar informe final y capturas de pantalla

Foto

Agregar foto

Ubicación

Usar ubicación actual

Completada

pantall

q w e r t y u i o p

a s d f g h j k l

z x c v b n m

?123 , . ←

TaskFormScreen — formulario de tarea con foto y ubicación

7:40

Mis tareas

Compartir

Compartir

Compartir

☐ Comprar materiales examen

☐ delectus aut autem

☐ quis ut nam facilis et officia qui

☐ fugiat veniam minus

☒ et porro tempora

☐ laboriosam mollitia et enim quasi adipi...

☐ qui ullam ratione quibusdam voluptate...

☐ illo expedita consequatur quia in

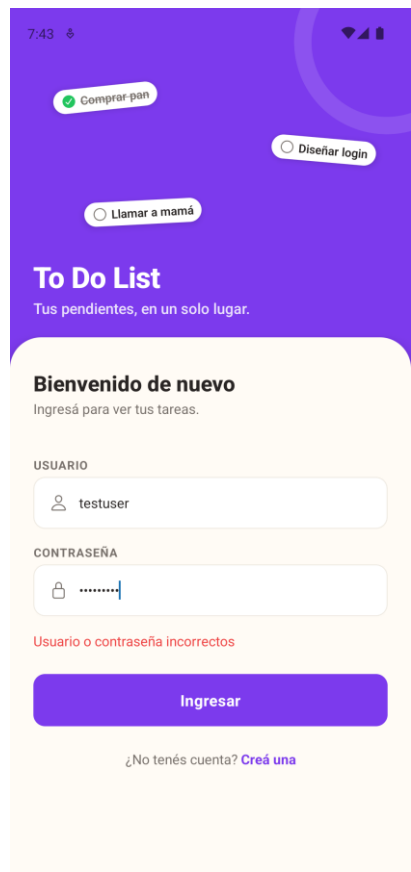
☒ que adipisci enim quam ut ab

☐ molestiae perspiciatis ipsa

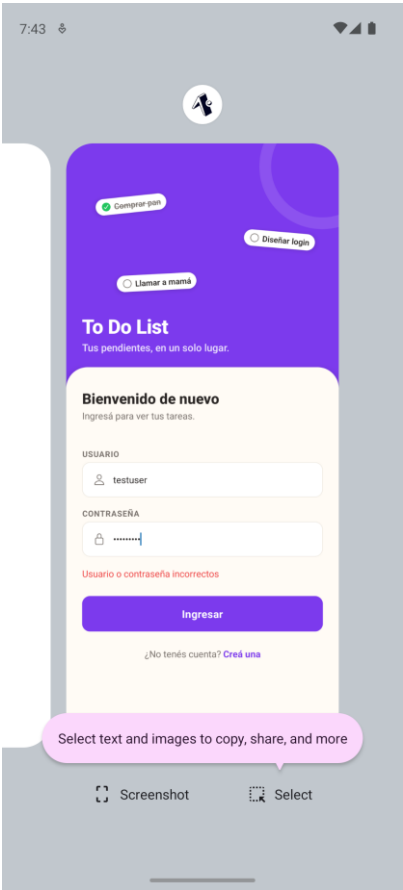
☒ illo est ratione doloremque quia maior...

+

TaskListScreen — lista de tareas, ícono de sincronizado



Login — validación de credenciales incorrectas



Ícono de la app instalada en el dispositivo

11. Pruebas automatizadas (Jest)

Framework: Jest con preset jest-expo, mockeando expo-camera y expo-location.

Archivo	Cobertura
TaskFormScreen-test.tsx	Captura de foto (compresión/guardado, photoUri seteado) y obtención de ubicación (coordenadas mockeadas, location seteado); guardado de tarea aun con permiso rechazado.
authStorage-test.ts	Alta de usuario, rechazo de usuario duplicado, login con credenciales inválidas.
jsonPlaceholder-test.ts	Integración con la API externa (importación de tareas).

Ejecución

```
cd
pnpm test
```

app

12. Instalación y ejecución

Requisitos previos

- Node.js
- pnpm (npm i -g pnpm)
- Expo Go instalada en un dispositivo Android (o emulador Android)

Pasos

```
cd
pnpm
pnpm start          # abre Metro / Expo Dev Tools, escanear QR con Expo Go
pnpm android        # abre directo en emulador/dispositivo Android
```

app
install

13. Roadmap (fuera de alcance actual)

- **Firebase Firestore:** reemplaza a JSONPlaceholder como backend remoto real, persiste escrituras y habilita sincronización.
- **Autenticación multi-dispositivo:** migrar de auth local a Firebase Authentication, sincronizando sesión y usuarios entre dispositivos.
- **Sincronización en tiempo real:** entre dispositivos, una vez migrado a Firestore.

14. Conclusión

El proyecto cumple los cuatro ejes solicitados: interacción con periféricos de hardware (cámara y GPS), autenticación de usuarios local con contraseñas hasheadas, integración con un servicio web externo (JSONPlaceholder, con importación y sincronización real vía POST) y pruebas automatizadas con Jest sobre los componentes de hardware.

La app funciona de forma completamente local — sin backend propio — usando AsyncStorage como fuente de verdad, dejando planificada la migración a Firebase Firestore como evolución futura fuera del alcance actual.

Los permisos de cámara y ubicación fueron probados en emulador Android: ambos siguen el flujo de solicitud lazy definido en el diseño, muestran el diálogo nativo correspondiente y, al ser otorgados, habilitan la funcionalidad asociada (captura y guardado de foto; obtención de ubicación, con una limitación conocida del emulador para la resolución final de la coordenada, ajena al código de la app).

Informe generado a partir de docs/BRIEF.md, docs/DESIGN.md, docs/PLAN.md, docs/pruebas/INFORME_PERMISOS.md y del código fuente del proyecto To Do List (com.ignac.todolist).